Rapport de Projet — Smart Lab (Web of Things)

Master Informatique — Parcours ISA | **Module :** Web/Data Services Based Integration | **Octobre 2026**

Introduction & Architecture du Système

Ce projet déploie un écosystème **Web of Things (WoT)** complet simulant un laboratoire intelligent distribué sans matériel physique, articulé autour des piliers **Access** → **Find** → **Share** → **Compose**. L'architecture repose sur **quatre microservices Spring Boot 3** autonomes communiquant via HTTP/REST, Webhooks et Server-Sent Events (SSE), complétés par un tableau de bord Web :

— **Passerelle (port 8080) :** Registre dynamique, mandataire inverse (*Auth Proxy*), moteur de règles d'automatisation (R1, R2), hub SSE et serveur statique du Dashboard.
— **Thermostat (`thing-thermostat`, port 8081) :** Simulation thermique continue (+0.2°C/s en chauffe, −0.05°C/s à l'arrêt), consigne `target`, modes `off/heat/eco`, émission de l'événement `targetReached`.
— **Lampe (`thing-lamp`, port 8082) :** Éclairage connecté, propriété booléenne `on`, gradateur `brightness` (0–100), actions `toggle` et `setBrightness`.
— **Capteur de Mouvement (`thing-motion`, port 8083) :** Horodatage de détection `lastMotion`, action `simulateMotion` et événement `motion`.

Le démarrage est unifié via `./run.sh` et l'ensemble des 38 tests d'intégration (`check-routes.sh`) est validé avec succès (100% conformité).

Access (Accès aux Ressources)

Le niveau **Access** standardise l'exposition et la manipulation des objets physiques sous forme de ressources Web RESTful légères (HTTP/JSON).

Modélisation des Affordances et Format des Échanges

Chaque objet structure ses interactions selon les trois affordances du Web of Things :

— **Propriétés (`/properties`, `/properties/{name}`)** : La lecture retourne obligatoirement `{"name": "...", "value": ...}`. L'écriture via `PUT` requiert `{"value": ...}`. Les propriétés en lecture seule (`temperature`, `lastMotion`) rejettent toute écriture par une erreur `400 Bad Request`.
— **Actions (`/actions/{name}`)** : Invoquées par `POST`. Elles acceptent un corps vide pour les actions sans paramètre (`toggle`, `simulateMotion`) ou un payload `{"value": ...}` (`setBrightness`, `setTarget`).
— **Événements** : Flux asynchrones émis par les objets lors de transitions d'état (`motion`, `targetReached`, `propertyChanged`).

Pattern *Auth Proxy*

Pour garantir le cloisonnement réseau et la sécurité, le Dashboard et les clients externes n'interrogent **jamais** les objets directement. Toutes les requêtes sont mandatées par la passerelle via les routes `/things/{id}/properties/**` et `/things/{id}/actions/**`. Le composant `ThingClient` relaye les requêtes à l'objet cible via `RestClient` (Spring Boot 3) avec des délais de garde stricts (2s connexion, 3s lecture). Si un objet est indisponible ou crashé, la passerelle intercepte l'erreur et retourne un statut normalisé `502 Bad Gateway`.

Codes Statuts HTTP et Format Uniforme des Erreurs

L'architecture implémente scrupuleusement la matrice des statuts de l'énoncé via un `@RestControllerAdvice` centralisé :

— **200 OK** : Lecture, écriture de propriété ou exécution d'action réussie.
— **201 Created** : Enregistrement d'un objet (avec en-tête obligatoire `Location: /things/{id}`).
— **202 Accepted** : Réception valide d'un webhook d'événement (`POST /events`).
— **204 No Content** : Désenregistrement propre d'un objet (`DELETE /things/{id}`).
— **400 Bad Request** : Champ manquant, valeur invalide/hors bornes (ex. luminosité à 150) ou écriture sur propriété en lecture seule.
— **401 Unauthorized** : Jeton Bearer absent ou invalide sur l'API de la passerelle.
— **403 Forbidden** : Tentative d'écriture avec un jeton en lecture seule (*rôle viewer*, bonus).
— **404 Not Found** : Objet, propriété ou action inconnu.
— **409 Conflict** : Enregistrement d'un équipement dont l'identifiant existe déjà.
— **502 Bad Gateway** : Équipement cible injoignable ou en timeout.

Toutes les erreurs renvoient systématiquement un payload explicite : `{"status": <code_http>, "error": "<message explicite>"}`.

Find (Découverte & Répertoire des Objets)

Le niveau **Find** permet la localisation dynamique et l'auto-description des objets connectés sans configuration préalable côté passerelle.

Auto-enregistrement Dynamique & Cycle de Vie

Au démarrage, chaque microservice d'objet invoque `POST /things` sur la passerelle avec son jeton et un corps décrivant son profil : `{"id": "...", "name": "...", "baseUrl": "...", "model": {...}}`. La passerelle stocke l'équipement dans une `ConcurrentHashMap<String, Thing>` thread-safe, renvoie `201 Created` avec l'en-tête `Location: /things/{id}`, et pousse un événement SSE de type `registry` synchronisant instantanément les navigateurs connectés. Si l'identifiant est déjà enregistré, la passerelle refuse l'inscription par un code `409 Conflict`. À l'arrêt du service, un hook `@PreDestroy` libère proprement la ressource via `DELETE /things/{id}` (`204 No Content`).

Auto-description : Web Thing Model

Chaque objet expose sa description sémantique sur `GET /model` selon le standard *Web Thing Model* du cours :

— Métadonnées : nom, description textuelle, version logicielle.
— Typage des propriétés : types primitifs (`boolean`, `number`, `string`), bornes de validité (`minimum`, `maximum`), mutabilité (`readOnly`).
— Schéma des actions : paramètres attendus et types d'entrée.
— Liens hypermédias : collection de liens relatifs vers les ressources (`"links": {"properties": "/properties", "toggle": "/actions/toggle", ...}`).

La consultation du catalogue s'effectue par `GET /things` (liste avec liens d'accès) et `GET /things/{id}` (fiche détaillée).

Share (Partage Sécurisé & Contrôle d'Accès)

Le niveau **Share** assure l'isolation et la protection des interfaces de programmation de la passerelle.

Authentification par Jeton Bearer

Toutes les requêtes vers la passerelle (`/things/**` et `/events/**`) doivent fournir l'en-tête HTTP : `Authorization: Bearer <token>`. Ce contrôle est intercepté par `TokenFilter` (`OncePerRequestFilter`). En cas d'omission ou de jeton invalide, l'accès est rejeté par un statut `401 Unauthorized` accompagné de l'en-tête `WWW-Authenticate: Bearer` et du corps JSON explicite. Les ressources statiques du Dashboard (`/`, `index.html`, CSS, JS) restent publiques pour permettre l'affichage de l'interface et la saisie utilisateur.

Contrôle d'Accès par Rôles : Opérateur vs Observateur (Bonus RBAC)

Un système de rôles a été implémenté pour enrichir la sécurité :

— **Rôle `operator` (`operator-secret`)** : Droits complets en lecture, écriture et déclenchement d'actions.
— **Rôle `viewer` (`viewer-secret`)** : Droits restreints en lecture seule. Les requêtes de lecture (`GET`) sont autorisées, mais toute tentative d'altération (`PUT`, `POST`, `DELETE`) est interceptée et rejetée par un statut `403 Forbidden` (`{"status":403, "error":"forbidden: read-only access"}`).

Streaming SSE Sécurisé

L'API navigateur `EventSource` ne supportant pas l'injection d'en-têtes HTTP personnalisés, une dérogation sécurisée permet la transmission du jeton via paramètre de requête pour le flux temps réel : `GET /events/stream?token=operator-secret`. Le filtre extrait et valide ce paramètre avec le même niveau d'exigence avant d'ouvrir la connexion persistante.

Contrat d'Interface OpenAPI 3.1

La totalité de l'API de la passerelle (endpoints, schémas JSON, codes HTTP et sécurité `bearerAuth`) est contractuellement spécifiée dans le fichier `gateway.yaml`, consultable et téléchargeable depuis le Dashboard.

Compose (Composition de Services & Automatisation)

Le niveau **Compose** orchestre les équipements autonomes au sein d'interactions automatisées transverses.

Architecture Événementielle : Webhooks & SSE vs WebSockets

Lorsqu'un changement d'état survient sur un objet (présence détectée, consigne de température atteinte), celui-ci poste un événement par Webhook vers la passerelle (`POST /events`). Pour prévenir tout blocage de boucle d'événements, l'émission est déportée dans un thread asynchrone (`CompletableFuture.runAsync`). À la réception, `EventHub` relaie l'événement en temps réel aux clients connectés via `GET /events/stream` (`SseEmitter`), en maintenant un *heartbeat ping* toutes les 15 secondes. **Justification du choix SSE vs WebSockets :** Le flux temps réel vers le client étant strictement unidirectionnel (diffusion serveur vers navigateur, les interactions ascendantes s'effectuant via REST), les Server-Sent Events s'imposent naturellement. SSE fonctionne sur HTTP standard sans surcoût de négociation full-duplex TCP, traverse nativement les pare-feux et gère automatiquement les reconnexions côté navigateur (`EventSource`).

Moteur de Règles d'Automatisation (`RulesEngine`)

La passerelle intègre un moteur événementiel implémentant les règles du scénario de référence :

— **Règle R1 (Éclairage intelligent réarmable) :** Sur réception de l'événement `motion`, la passerelle allume la lampe par écriture directe de la propriété : `PUT http://localhost:8082/properties/on` (`{"value": true}`). Conformément aux exigences du sujet, l'action `toggle` n'est **jamais** utilisée afin d'éviter d'éteindre une lampe déjà allumée. Un minuteur réarmable (`RestartableTimer` s'appuyant sur `TaskScheduler`) est initialisé pour un délai T_1 (10s en démo). Tout nouveau mouvement annule la planification antérieure et relance le décompte. À expiration sans mouvement, la lampe est éteinte (`PUT /properties/on` à `false`).
— **Règle R2 (Confort thermique multi-conditions) :** Sur réception de `motion`, la passerelle interroge la température actuelle du thermostat. Si température < 19.0°C, elle enclenche la chauffe en basculant le mode à `heat` et la consigne à 19.0°C (`target = 19.0`). Lorsque le thermostat atteint 19.0°C, il émet l'événement `targetReached`. Si le thermostat est en mode `heat`, la passerelle coupe immédiatement le chauffage en basculant le mode à `off`. En mode `eco`, cet événement est ignoré.
— **Ordre d'application & Isolation :** Sur détection de mouvement, l'ordre d'évaluation strict est : fin de R3 (si mode éco actif), puis R1, puis R2. Les règles appellent les objets **directement sur leurs ports dédiés** sans transiter par le proxy de la passerelle, garantissant que ces écritures automatiques ne soient pas interprétées comme des commandes manuelles (condition requise pour la règle de débrayage R4).

Moteur de Simulation Thermique

Le thermostat embarque une tâche planifiée cadencée à 1 Hz (`@Scheduled(fixedRate = 1000)`) :

— En modes `heat` et `eco`, la température s'élève de $+0.2$°C/s vers la consigne. À la première atteinte de consigne par le bas, l'événement `targetReached` est émis une seule fois (protégé par un drapeau atomique), puis la température est maintenue stable.
— En mode `off`, la température dérive lentement vers la température extérieure (15.0°C) à raison de -0.05°C/s, sans émission d'événement.
— La température initiale est fixée à 17.5°C au démarrage pour permettre la démonstration immédiate de la règle R2.

Limits and possible improvements (Limites & Améliorations)

Bien que le système valide 100% des tests fonctionnels et d'intégration, plusieurs axes d'évolution architecturale peuvent être dégagés pour un contexte industriel :

— **Persistance de l'état (actuellement volatile en mémoire) :** Le répertoire des objets et les données d'état reposent sur des maps mémoires. L'intégration d'une base légère ou d'un cache distribué permettrait la résilience en cas de redémarrage.
— **Sonde de disponibilité proactive (*Liveness Checking*, bonus) :** La passerelle ne détecte actuellement la coupure d'un objet qu'à la sollicitation (502 Bad Gateway). Une tâche périodique interrogeant `GET /model` toutes les 10s permettrait d'actualiser dynamiquement un champ `"status": "online"|"offline"` dans `GET /things`.
— **Implémentation des règles R3 et R4 (bonus) :** *R3 (Économie d'énergie) :* Déclenchement d'un timer T_3 après inactivité prolongée basculant le thermostat en mode `eco` (17°C). *R4 (Priorité utilisateur / Débrayage manuel) :* Détection des écritures proxy initiées par le dashboard pour suspendre l'automatisation durant T_4 ou jusqu'à reprise explicite (`POST /things/{id}/automation/resume`).
— **Format sémantique W3C Thing Description (TD 1.1) :** Faire évoluer le format descriptif JSON du modèle vers du JSON-LD conforme au W3C WoT Thing Description avec vocabulaire SAREF.
— **Sécurisation mutuelle passerelle-objets (*Thing Gateway Key*) :** Restreindre les microservices des objets pour qu'ils n'acceptent de requêtes qu'en provenance de la passerelle (clé secrète d'interconnexion ou mTLS).